Assignment 5

Analyze

A). The two AI programs I used were:

- Claude Sonnet 5.5 Medium
- Gemini 3.0 Models

B). I accessed both programs on my computer through Google Chrome.

C). Prompt:

I have an existing C++ Sudoku solver for a software engineering assignment. I need to modify the program rather than replace it entirely.

Please modify the attached code to meet these requirements:

1. Keep all original code functionality working.
2. Read the five input files: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
3. Write all valid solutions for each puzzle to a corresponding file named solutionN.txt, where N is the puzzle number.
4. If no solutions exist, write exactly "No solution found." to the corresponding output file.
5. If solutions exist, write every solution followed by a final line containing "Solutions found: n", where n is the total number of solutions.
6. Format the solutions the same way as the original input puzzle files.
7. Preserve the original program's structure and functions wherever possible.
8. Make the code compatible with C++11 on a Linux server.

Provide the complete modified C++ code and explain the changes you made.

Along with the given cpp file from another student.

D). GenAI Code

Geminis Code:

```
/*
 * Program: EECS 348 Assignment 4 (Modified)
 * Description: Reads and solves five Sudoku puzzles using recursive backtracking,
 *              writing results to corresponding solution files.
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt.
 * Outputs: solution1.txt, solution2.txt, solution3.txt, solution4.txt, solution5.txt.
 * Author: Prabesh Sapkota
 *Collaborators/Editors: Conor Rothe
 */

#include <iostream> // Provides console input and output.
#include <fstream>  // Provides file-reading and writing tools.
#include <string>   // Provides the string type and to_string.
#include <vector>   // Stores and validates the puzzle's 81 input cells.
#include <cctype>   // Provides the isspace function.
using namespace std; // Allows standard-library names without the std:: prefix.
```

```
class SudokuSolver { // Defines a class that stores and solves one Sudoku puzzle.
private: // Begins the private section of the class.
    int board[9][9]; // Stores the Sudoku grid; zero represents an empty cell.
    int solutionCount; // Counts how many complete solutions have been found.

    bool isSafe(int row, int col, int digit) { // Checks whether digit can go in the spec
ified cell.
        for (int j = 0; j < 9; j++) { // Checks all cells in the selected row.
            if (board[row][j] == digit) { // Detects the same digit in that row.
                return false; // Rejects a digit that already appears in the row.
            } // Ends the row-conflict condition.
        } // Ends the row scan.

        for (int i = 0; i < 9; i++) { // Checks all cells in the selected column.
            if (board[i][col] == digit) { // Detects the same digit in that column.
                return false; // Rejects a digit that already appears in the column.
            } // Ends the column-conflict condition.
        } // Ends the column scan.

        int startRow = (row / 3) * 3; // Finds the first row of the cell's 3-by-3 block.
        int startCol = (col / 3) * 3; // Finds the first column of the cell's 3-by-3 bloc
k.

        for (int i = startRow; i < startRow + 3; i++) { // Visits the three rows of the b
lock.
            for (int j = startCol; j < startCol + 3; j++) { // Visits the three columns o
f the block.
                if (board[i][j] == digit) { // Detects the same digit inside the block.
                    return false; // Rejects a digit that already appears in the block.
                } // Ends the block-conflict condition.
            } // Ends the inner block scan.
        } // Ends the outer block scan.

        return true; // Confirms that the digit follows all Sudoku placement rules.
    } // Ends isSafe.

    void printBoard(ostream& out = cout) const { // Prints/writes the Sudoku grid to the
specified output stream.
        for (int i = 0; i < 9; i++) { // Visits each row of the board.
            if (i % 3 == 0) { // Checks whether a new group of three rows begins.
                out << "+-------+-------+-------+" << endl; // Prints the horizontal grid
border.
            } // Ends the row-group condition.

            for (int j = 0; j < 9; j++) { // Visits each cell in the current row.
                if (j % 3 == 0) { // Checks whether a new group of three columns begins.
                    out << "| "; // Prints the left boundary of a 3-column group.
                } // Ends the column-group condition.

                if (board[i][j] == 0) { // Checks whether the current cell is empty.
```

```
                    out << "_ "; // Displays an underscore for an empty cell.
                } else { // Handles a cell containing a digit.
                    out << board[i][j] << " "; // Displays the digit stored in the cell.
                } // Ends the empty/non-empty condition.
            } // Ends the scan of the current row.

            out << "|" << endl; // Prints the right boundary and starts a new output lin
e.
        } // Ends the scan of all rows.

        out << "+-------+-------+-------+" << endl; // Prints the bottom border of the gr
id.
    } // Ends printBoard.

    bool findBestEmptyCell(int& bestRow, int& bestCol) { // Finds an empty cell with the
fewest legal candidates.
        int fewestCandidates = 10; // Starts above the maximum possible candidate count o
f nine.
        bestRow = -1; // Indicates that no empty cell has been selected yet.
        bestCol = -1; // Indicates that no empty cell has been selected yet.

        for (int row = 0; row < 9; row++) { // Searches each row of the board.
            for (int col = 0; col < 9; col++) { // Searches each column of the current ro
w.
                if (board[row][col] != 0) { // Skips cells that already contain a digit.
                    continue; // Moves on to the next cell.
                } // Ends the non-empty-cell condition.

                int candidateCount = 0; // Counts legal digits for this empty cell.

                for (int digit = 1; digit <= 9; digit++) { // Tries each Sudoku digit as
a candidate.
                    if (isSafe(row, col, digit)) { // Checks whether the candidate obeys
Sudoku rules.
                        candidateCount++; // Counts this legal candidate.
                    } // Ends the candidate-validity condition.
                } // Ends the candidate-counting loop.

                if (candidateCount == 0) { // Detects an empty cell with no legal candida
te.
                    bestRow = row; // Saves the contradictory cell's row.
                    bestCol = col; // Saves the contradictory cell's column.
                    return true; // Signals that an empty cell exists, but it has no poss
ible value.
                } // Ends the no-candidate condition.

                if (candidateCount < fewestCandidates) { // Checks whether this cell has
fewer candidates than the best so far.
                    fewestCandidates = candidateCount; // Records the new lowest candidat
e count.
                    bestRow = row; // Records the row of the best candidate cell.
```

```
                    bestCol = col; // Records the column of the best candidate cell.

                    if (fewestCandidates == 1) { // Checks whether this cell has only one
possible digit.
                        return true; // A single-candidate cell cannot be improved upon.
                    } // Ends the single-candidate condition.
                } // Ends the best-cell update condition.
            } // Ends the scan of the current row.
        } // Ends the scan of the board.

        return bestRow != -1; // Returns true if an empty cell was found, otherwise the b
oard is complete.
    } // Ends findBestEmptyCell.

    void solveRecursive(ostream& out) { // Searches recursively for all possible Sudoku s
olutions, writing to stream.
        int emptyRow = -1; // Holds the row of the selected empty cell.
        int emptyCol = -1; // Holds the column of the selected empty cell.

        if (!findBestEmptyCell(emptyRow, emptyCol)) { // Checks whether there are no empt
y cells left.
            solutionCount++; // Counts the completed valid board as a solution.
            out << "\nSolution " << solutionCount << ":" << endl; // Prints the solution
number.
            printBoard(out); // Displays the completed Sudoku solution to the stream.
            return; // Backtracks to search for any additional solutions.
        } // Ends the completed-board condition.

        if (emptyRow == -1 || emptyCol == -1) { // Checks for an invalid selection state
defensively.
            return; // Stops this branch if no valid cell coordinates were selected.
        } // Ends the defensive coordinate check.

        int candidateCount = 0; // Counts legal values for the selected cell.

        for (int digit = 1; digit <= 9; digit++) { // Tests each digit for the selected c
ell.
            if (isSafe(emptyRow, emptyCol, digit)) { // Checks whether the digit is legal
in this cell.
                candidateCount++; // Records that at least one valid candidate exists.
                board[emptyRow][emptyCol] = digit; // Places the candidate digit in the b
oard.
                solveRecursive(out); // Solves the remaining empty cells using recursion.
                board[emptyRow][emptyCol] = 0; // Removes the candidate to try another po
ssibility.
            } // Ends the valid-candidate condition.
        } // Ends the digit-trial loop.

        if (candidateCount == 0) { // Identifies a branch where no digit can fit the sele
cted cell.
            return; // Abandons this branch so the previous recursive call can backtrack.
```

```
        } // Ends the dead-end condition.
    } // Ends solveRecursive.

public: // Begins the public section of the class.
    SudokuSolver() : solutionCount(0) { // Initializes the solution counter when an objec
t is created.
        for (int row = 0; row < 9; row++) { // Visits each row during initialization.
            for (int col = 0; col < 9; col++) { // Visits each column during initializati
on.
                board[row][col] = 0; // Marks every cell as empty.
            } // Ends the initialization of one row.
        } // Ends board initialization.
    } // Ends the constructor.

    bool loadPuzzle(const string& filename) { // Loads and validates the basic format of
a puzzle file.
        ifstream inputFile(filename); // Opens the specified file for reading.

        if (!inputFile.is_open()) { // Checks whether the file opened successfully.
            cerr << "Error: Could not open " << filename << endl; // Reports a missing or
inaccessible file.
            return false; // Stops loading when the file cannot be opened.
        } // Ends the file-open check.

        vector<char> cells; // Temporarily stores all 81 cell symbols before changing the
board.
        char ch; // Stores each character read from the file.

        while (inputFile.get(ch)) { // Reads the file one character at a time.
            if ((ch >= '1' && ch <= '9') || ch == '_') { // Accepts digits one through ni
ne and underscores.
                cells.push_back(ch); // Saves a valid Sudoku cell symbol.
            } else if (ch == '0') { // Allows zero as another way to mark an empty cell.
                cells.push_back('_'); // Normalizes zero to the underscore blank marker.
            } else if (!isspace(static_cast<unsigned char>(ch))) { // Rejects unexpected
non-whitespace characters.
                cerr << "Error: Invalid character in " << filename << endl; // Reports an
unexpected character.
                return false; // Stops loading a malformed puzzle file.
            } // Ends the character-validation branches.
        } // Ends the file-reading loop.

        if (cells.size() != 81) { // Requires exactly 81 cell symbols for a 9-by-9 puzzl
e.
            cerr << "Error: " << filename << " must contain exactly 81 Sudoku cells." <<
endl; // Reports the incorrect cell count.
            return false; // Rejects a puzzle with too few or too many cells.
        } // Ends the cell-count check.

        for (int row = 0; row < 9; row++) { // Copies the validated symbols into each boa
rd row.
```

```
            for (int col = 0; col < 9; col++) { // Copies the symbols into each board col
umn.
                char cell = cells[row * 9 + col]; // Retrieves the symbol at the current
linear position.

                if (cell == '_') { // Checks whether the symbol represents a blank cell.
                    board[row][col] = 0; // Stores blank cells internally as zero.
                } else { // Handles a symbol containing a digit.
                    board[row][col] = cell - '0'; // Converts the digit character into it
s integer value.
                } // Ends the cell-conversion condition.
            } // Ends copying one row.
        } // Ends copying all 81 cells.

        return true; // Confirms that the file was read and formatted correctly.
    } // Ends loadPuzzle.

    bool isInitialBoardValid() { // Checks that the puzzle's original digits do not confl
ict.
        for (int row = 0; row < 9; row++) { // Examines each row of the starting puzzle.
            for (int col = 0; col < 9; col++) { // Examines each cell in the current row.
                if (board[row][col] == 0) { // Checks whether the current cell is blank.
                    continue; // Skips blank cells because they cannot conflict with a gi
ven digit.
                } // Ends the blank-cell condition.

                int digit = board[row][col]; // Saves the digit currently being checked.
                board[row][col] = 0; // Temporarily clears the cell so it does not confli
ct with itself.
                bool valid = isSafe(row, col, digit); // Checks the saved digit against i
ts row, column, and block.
                board[row][col] = digit; // Restores the original digit after checking i
t.

                if (!valid) { // Checks whether another given digit conflicts with this o
ne.
                    return false; // Rejects the starting puzzle if a conflict is found.
                } // Ends the conflict condition.
            } // Ends the scan of the current row.
        } // Ends the scan of the starting board.

        return true; // Confirms that all given digits are mutually consistent.
    } // Ends isInitialBoardValid.

    void solve(const string& inFilename, const string& outFilename) { // Solves puzzle an
d writes output to file.
        ofstream outFile(outFilename); // Opens the corresponding solution file for writi
ng.
        if (!outFile.is_open()) {
            cerr << "Error: Could not open output file " << outFilename << endl;
            return;
```

```
        }

        // Maintains console output for tracking and verification (original functionalit
y)
        cout << "\n========================================" << endl;
        cout << "Puzzle file: " << inFilename << endl;
        cout << "========================================" << endl;

        cout << "\nOriginal puzzle:" << endl;
        printBoard(cout);

        if (!isInitialBoardValid()) {
            cout << "\nNo solution found: the original puzzle has conflicting digits." <<
endl;
            outFile << "No solution found." << endl; // Exactly matches required error st
ring
            outFile.close();
            return;
        }

        solutionCount = 0;
        solveRecursive(outFile); // Recursively solves and writes solutions to the output
file

        if (solutionCount == 0) {
            cout << "\nNo solution found." << endl;
            outFile << "No solution found." << endl; // Exactly matches requirement 4
        } else {
            cout << "\nTotal solutions found: " << solutionCount << endl;
            outFile << "Solutions found: " << solutionCount << endl; // Exactly matches r
equirement 5
        }

        outFile.close(); // Closes the output file
    } // Ends solve.
}; // Ends the SudokuSolver class definition.

int main() { // Defines the program's starting point.
    // Loops through the five required input files and generates corresponding output fil
es
    for (int i = 1; i <= 5; i++) {
        string inFilename = "puzzle" + to_string(i) + ".txt";
        string outFilename = "solution" + to_string(i) + ".txt";

        SudokuSolver solver; // Creates a fresh solver so puzzles do not share board stat
e.

        if (solver.loadPuzzle(inFilename)) { // Loads the current file and continues if v
alid.
            solver.solve(inFilename, outFilename); // Solves and writes output to solutio
nN.txt.
```

```
        }
    }

    return 0; // Reports successful completion of the program.
} // Ends main.
```

Claudes Code:

```
/*
 * Program: EECS 348 Assignment 4
 * Description: Reads and solves five Sudoku puzzles using recursive backtracking.
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt.
 * Output: Console display of each puzzle and its solutions, plus solution1.txt through s
olution5.txt.
 * Collaborators/Editors: Conor Rothe
 * Other sources: none
 * Author: Prabesh Sapkota
 * Creation Date: October 15, 2026
 */

#include <iostream> // Provides console input and output.
#include <fstream> // Provides file-reading tools.
#include <string> // Provides the string type.
#include <vector> // Stores and validates the puzzle's 81 input cells.
#include <cctype> // Provides the isspace function.
using namespace std; // Allows standard-library names without the std:: prefix.

class SudokuSolver { // Defines a class that stores and solves one Sudoku puzzle.
private: // Begins the private section of the class.
    int board[9][9]; // Stores the Sudoku grid; zero represents an empty cell.
    int solutionCount; // Counts how many complete solutions have been found.
    ofstream outputFile; // Writes each solution to the current puzzle's solutionN.txt fi
le.

    bool isSafe(int row, int col, int digit) { // Checks whether digit can go in the spec
ified cell.
        for (int j = 0; j < 9; j++) { // Checks all cells in the selected row.
            if (board[row][j] == digit) { // Detects the same digit in that row.
                return false; // Rejects a digit that already appears in the row.
            } // Ends the row-conflict condition.
        } // Ends the row scan.

        for (int i = 0; i < 9; i++) { // Checks all cells in the selected column.
            if (board[i][col] == digit) { // Detects the same digit in that column.
                return false; // Rejects a digit that already appears in the column.
            } // Ends the column-conflict condition.
        } // Ends the column scan.

        int startRow = (row / 3) * 3; // Finds the first row of the cell's 3-by-3 block.
        int startCol = (col / 3) * 3; // Finds the first column of the cell's 3-by-3 bloc
k.
```

```
        for (int i = startRow; i < startRow + 3; i++) { // Visits the three rows of the b
lock.
            for (int j = startCol; j < startCol + 3; j++) { // Visits the three columns o
f the block.
                if (board[i][j] == digit) { // Detects the same digit inside the block.
                    return false; // Rejects a digit that already appears in the block.
                } // Ends the block-conflict condition.
            } // Ends the inner block scan.
        } // Ends the outer block scan.

        return true; // Confirms that the digit follows all Sudoku placement rules.
    } // Ends isSafe.

    void printBoard() const { // Prints the current Sudoku grid without modifying it.
        for (int i = 0; i < 9; i++) { // Visits each row of the board.
            if (i % 3 == 0) { // Checks whether a new group of three rows begins.
                cout << "+-------+-------+-------+" << endl; // Prints the horizontal gri
d border.
            } // Ends the row-group condition.

            for (int j = 0; j < 9; j++) { // Visits each cell in the current row.
                if (j % 3 == 0) { // Checks whether a new group of three columns begins.
                    cout << "| "; // Prints the left boundary of a 3-column group.
                } // Ends the column-group condition.

                if (board[i][j] == 0) { // Checks whether the current cell is empty.
                    cout << "_ "; // Displays an underscore for an empty cell.
                } else { // Handles a cell containing a digit.
                    cout << board[i][j] << " "; // Displays the digit stored in the cell.
                } // Ends the empty/non-empty condition.
            } // Ends the scan of the current row.

            cout << "|" << endl; // Prints the right boundary and starts a new output lin
e.
        } // Ends the scan of all rows.

        cout << "+-------+-------+-------+" << endl; // Prints the bottom border of the g
rid.
    } // Ends printBoard.

    void writeBoardToFile() { // Writes the current board to the output file in the input
-file format.
        for (int i = 0; i < 9; i++) { // Visits each row of the board.
            for (int j = 0; j < 9; j++) { // Visits each cell in the current row.
                if (j > 0) { // Checks whether a separator is needed before this cell.
                    outputFile << " "; // Separates cells with a single space, as in the
input files.
                } // Ends the separator condition.

                if (board[i][j] == 0) { // Checks whether the current cell is empty.
```

```
                    outputFile << "_"; // Writes an underscore for an empty cell, matchin
g the input format.
                } else { // Handles a cell containing a digit.
                    outputFile << board[i][j]; // Writes the digit stored in the cell.
                } // Ends the empty/non-empty condition.
            } // Ends the scan of the current row.

            outputFile << "\n"; // Ends the current row of the output file.
        } // Ends the scan of all rows.
    } // Ends writeBoardToFile.

    bool findBestEmptyCell(int& bestRow, int& bestCol) { // Finds an empty cell with the
fewest legal candidates.
        int fewestCandidates = 10; // Starts above the maximum possible candidate count o
f nine.
        bestRow = -1; // Indicates that no empty cell has been selected yet.
        bestCol = -1; // Indicates that no empty cell has been selected yet.

        for (int row = 0; row < 9; row++) { // Searches each row of the board.
            for (int col = 0; col < 9; col++) { // Searches each column of the current ro
w.
                if (board[row][col] != 0) { // Skips cells that already contain a digit.
                    continue; // Moves on to the next cell.
                } // Ends the non-empty-cell condition.

                int candidateCount = 0; // Counts legal digits for this empty cell.

                for (int digit = 1; digit <= 9; digit++) { // Tries each Sudoku digit as
a candidate.
                    if (isSafe(row, col, digit)) { // Checks whether the candidate obeys
Sudoku rules.
                        candidateCount++; // Counts this legal candidate.
                    } // Ends the candidate-validity condition.
                } // Ends the candidate-counting loop.

                if (candidateCount == 0) { // Detects an empty cell with no legal candida
te.
                    bestRow = row; // Saves the contradictory cell's row.
                    bestCol = col; // Saves the contradictory cell's column.
                    return true; // Signals that an empty cell exists, but it has no poss
ible value.
                } // Ends the no-candidate condition.

                if (candidateCount < fewestCandidates) { // Checks whether this cell has
fewer candidates than the best so far.
                    fewestCandidates = candidateCount; // Records the new lowest candidat
e count.
                    bestRow = row; // Records the row of the best candidate cell.
                    bestCol = col; // Records the column of the best candidate cell.

                    if (fewestCandidates == 1) { // Checks whether this cell has only one
```

```
possible digit.
                        return true; // A single-candidate cell cannot be improved upon.
                    } // Ends the single-candidate condition.
                } // Ends the best-cell update condition.
            } // Ends the scan of the current row.
        } // Ends the scan of the board.

        return bestRow != -1; // Returns true if an empty cell was found, otherwise the b
oard is complete.
    } // Ends findBestEmptyCell.

    void solveRecursive() { // Searches recursively for all possible Sudoku solutions.
        int emptyRow = -1; // Holds the row of the selected empty cell.
        int emptyCol = -1; // Holds the column of the selected empty cell.

        if (!findBestEmptyCell(emptyRow, emptyCol)) { // Checks whether there are no empt
y cells left.
            solutionCount++; // Counts the completed valid board as a solution.
            cout << "\nSolution " << solutionCount << ":" << endl; // Prints the solution
number.
            printBoard(); // Displays the completed Sudoku solution.

            if (outputFile.is_open()) { // Writes to the file only if it was opened succe
ssfully.
                if (solutionCount > 1) { // Checks whether an earlier solution precedes t
his one.
                    outputFile << "\n"; // Separates consecutive solutions with a blank l
ine.
                } // Ends the separator condition.
                writeBoardToFile(); // Saves the completed solution to the output file.
            } // Ends the file-output condition.
            return; // Backtracks to search for any additional solutions.
        } // Ends the completed-board condition.

        if (emptyRow == -1 || emptyCol == -1) { // Checks for an invalid selection state
defensively.
            return; // Stops this branch if no valid cell coordinates were selected.
        } // Ends the defensive coordinate check.

        int candidateCount = 0; // Counts legal values for the selected cell.

        for (int digit = 1; digit <= 9; digit++) { // Tests each digit for the selected c
ell.
            if (isSafe(emptyRow, emptyCol, digit)) { // Checks whether the digit is legal
in this cell.
                candidateCount++; // Records that at least one valid candidate exists.
                board[emptyRow][emptyCol] = digit; // Places the candidate digit in the b
oard.
                solveRecursive(); // Solves the remaining empty cells using recursion.
                board[emptyRow][emptyCol] = 0; // Removes the candidate to try another po
ssibility.
```

```
            } // Ends the valid-candidate condition.
        } // Ends the digit-trial loop.

        if (candidateCount == 0) { // Identifies a branch where no digit can fit the sele
cted cell.
            return; // Abandons this branch so the previous recursive call can backtrack.
        } // Ends the dead-end condition.
    } // Ends solveRecursive.

public: // Begins the public section of the class.
    SudokuSolver() : solutionCount(0) { // Initializes the solution counter when an objec
t is created.
        for (int row = 0; row < 9; row++) { // Visits each row during initialization.
            for (int col = 0; col < 9; col++) { // Visits each column during initializati
on.
                board[row][col] = 0; // Marks every cell as empty.
            } // Ends the initialization of one row.
        } // Ends board initialization.
    } // Ends the constructor.

    bool loadPuzzle(const string& filename) { // Loads and validates the basic format of
a puzzle file.
        ifstream inputFile(filename); // Opens the specified file for reading.

        if (!inputFile.is_open()) { // Checks whether the file opened successfully.
            cerr << "Error: Could not open " << filename << endl; // Reports a missing or
inaccessible file.
            return false; // Stops loading when the file cannot be opened.
        } // Ends the file-open check.

        vector<char> cells; // Temporarily stores all 81 cell symbols before changing the
board.
        char ch; // Stores each character read from the file.

        while (inputFile.get(ch)) { // Reads the file one character at a time.
            if ((ch >= '1' && ch <= '9') || ch == '_') { // Accepts digits one through ni
ne and underscores.
                cells.push_back(ch); // Saves a valid Sudoku cell symbol.
            } else if (ch == '0') { // Allows zero as another way to mark an empty cell.
                cells.push_back('_'); // Normalizes zero to the underscore blank marker.
            } else if (!isspace(static_cast<unsigned char>(ch))) { // Rejects unexpected
non-whitespace characters.
                cerr << "Error: Invalid character in " << filename << endl; // Reports an
unexpected character.
                return false; // Stops loading a malformed puzzle file.
            } // Ends the character-validation branches.
        } // Ends the file-reading loop.

        if (cells.size() != 81) { // Requires exactly 81 cell symbols for a 9-by-9 puzzl
e.
            cerr << "Error: " << filename << " must contain exactly 81 Sudoku cells." <<
```

```
endl; // Reports the incorrect cell count.
            return false; // Rejects a puzzle with too few or too many cells.
        } // Ends the cell-count check.

        for (int row = 0; row < 9; row++) { // Copies the validated symbols into each boa
rd row.
            for (int col = 0; col < 9; col++) { // Copies the symbols into each board col
umn.
                char cell = cells[row * 9 + col]; // Retrieves the symbol at the current
linear position.

                if (cell == '_') { // Checks whether the symbol represents a blank cell.
                    board[row][col] = 0; // Stores blank cells internally as zero.
                } else { // Handles a symbol containing a digit.
                    board[row][col] = cell - '0'; // Converts the digit character into it
s integer value.
                } // Ends the cell-conversion condition.
            } // Ends copying one row.
        } // Ends copying all 81 cells.

        return true; // Confirms that the file was read and formatted correctly.
    } // Ends loadPuzzle.

    bool isInitialBoardValid() { // Checks that the puzzle's original digits do not confl
ict.
        for (int row = 0; row < 9; row++) { // Examines each row of the starting puzzle.
            for (int col = 0; col < 9; col++) { // Examines each cell in the current row.
                if (board[row][col] == 0) { // Checks whether the current cell is blank.
                    continue; // Skips blank cells because they cannot conflict with a gi
ven digit.
                } // Ends the blank-cell condition.

                int digit = board[row][col]; // Saves the digit currently being checked.
                board[row][col] = 0; // Temporarily clears the cell so it does not confli
ct with itself.
                bool valid = isSafe(row, col, digit); // Checks the saved digit against i
ts row, column, and block.
                board[row][col] = digit; // Restores the original digit after checking i
t.

                if (!valid) { // Checks whether another given digit conflicts with this o
ne.
                    return false; // Rejects the starting puzzle if a conflict is found.
                } // Ends the conflict condition.
            } // Ends the scan of the current row.
        } // Ends the scan of the starting board.

        return true; // Confirms that all given digits are mutually consistent.
    } // Ends isInitialBoardValid.

    void writeNoSolution(const string& outputFilename) { // Writes the required no-soluti
```

```
on message to a file.
        ofstream failFile(outputFilename); // Opens (and overwrites) the specified output
file.

        if (!failFile.is_open()) { // Checks whether the output file opened successfully.
            cerr << "Error: Could not write " << outputFilename << endl; // Reports an ou
tput-file failure.
            return; // Stops because nothing can be written.
        } // Ends the file-open check.

        failFile << "No solution found." << endl; // Writes exactly the required message.
    } // Ends writeNoSolution.

    void solve(const string& filename, const string& outputFilename) { // Prints a puzzl
e, searches for all its solutions, and saves them.
        cout << "\n========================================" << endl; // Prints a separat
or before the puzzle heading.
        cout << "Puzzle file: " << filename << endl; // Identifies the file being process
ed.
        cout << "========================================" << endl; // Prints a separator
after the puzzle heading.

        cout << "\nOriginal puzzle:" << endl; // Labels the original, unsolved puzzle.
        printBoard(); // Displays the board as it was loaded from the file.

        if (!isInitialBoardValid()) { // Checks for conflicting given digits before searc
hing.
            cout << "\nNo solution found: the original puzzle has conflicting digits." <<
endl; // Explains why solving was stopped.
            writeNoSolution(outputFilename); // Records the failure in the output file.
            return; // Skips backtracking for an invalid starting board.
        } // Ends the original-board validation condition.

        solutionCount = 0; // Resets the solution count before this search begins.
        outputFile.open(outputFilename.c_str()); // Opens the solution file, overwriting
any previous contents.

        if (!outputFile.is_open()) { // Checks whether the output file opened successfull
y.
            cerr << "Error: Could not write " << outputFilename << endl; // Reports an ou
tput-file failure.
        } // Ends the output-open check.

        solveRecursive(); // Starts recursive backtracking with the selected-cell heurist
ic.

        if (solutionCount == 0) { // Checks whether the search found no complete solutio
n.
            cout << "\nNo solution found." << endl; // Reports that no valid solution exi
sts.
```

```
            if (outputFile.is_open()) { // Writes the message only if the file is open.
                outputFile << "No solution found." << endl; // Writes exactly the require
d message.
            } // Ends the file-output condition.
        } else { // Handles a puzzle for which one or more solutions were found.
            cout << "\nTotal solutions found: " << solutionCount << endl; // Displays the
number of solutions found.

            if (outputFile.is_open()) { // Writes the total only if the file is open.
                outputFile << "\nSolutions found: " << solutionCount << endl; // Writes t
he required final count line.
            } // Ends the file-output condition.
        } // Ends the solution-result condition.

        if (outputFile.is_open()) { // Checks whether the output file is still open.
            outputFile.close(); // Closes the file so all data is flushed to disk.
        } // Ends the close condition.
    } // Ends solve.
}; // Ends the SudokuSolver class definition.

int main() { // Defines the program's starting point.
    const string filenames[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.tx
t", "puzzle5.txt"}; // Lists the five required puzzle files.

    for (int i = 0; i < 5; i++) { // Processes each required puzzle number in sequence.
        const string& filename = filenames[i]; // Selects the current input filename.
        string outputFilename = "solution" + to_string(i + 1) + ".txt"; // Builds the mat
ching solutionN.txt name.
        SudokuSolver solver; // Creates a fresh solver so puzzles do not share board stat
e.

        if (solver.loadPuzzle(filename)) { // Loads the current file and continues only i
f its format is valid.
            solver.solve(filename, outputFilename); // Validates, solves, and saves the l
oaded puzzle.
        } else { // Handles a missing or malformed puzzle file.
            solver.writeNoSolution(outputFilename); // Records that no solution exists fo
r this input.
        } // Ends the load-success condition.
    } // Ends processing of the five puzzle files.

    return 0; // Reports successful completion of the program.
} // Ends main.
```

1. Correctness

Gemini: Gemini uses recursive backtracking, checks that the starting puzzle is valid, and makes sure the input has exactly 81 cells. It searches for all possible solutions. One issue is that it writes the solutions with grid borders and solution labels, which do not match the format of the original puzzle files. It also does not create an output file if an input file cannot be loaded.

Claude: Claude uses the same solving method and input checks, but it writes each solution in a format closer to the original puzzle files. It also creates an output file when a puzzle is missing or invalid and keeps the console output from the original program. Overall, Claude seems to meet the assignment requirements more closely.

2. Time Complexity

Both programs use recursive backtracking, so the worst-case search time is roughly O(9^E), where E is the number of empty cells. They also use the minimum-remaining-values approach: the solver picks the empty cell with the fewest legal digits first, which can cut down on extra searching. Gemini writes solutions to a file, while Claude prints them to the console and saves them to a file. Claude may do a little more output work, but the two programs have the same overall time complexity.

3. Space Complexity

Both programs use a fixed 9×9 board and do not save every solution in memory. Their memory use stays small for a standard Sudoku puzzle, although the recursive calls use additional space based on how many cells need to be filled. Gemini passes an output stream through its recursive function, while Claude keeps the output file stream in the solver class. Since the board size is fixed, both are about equally memory-efficient for this assignment.

4. Maintainability

Gemini: Gemini keeps most of the original program and passes an output stream to write solutions. This keeps the changes fairly simple and avoids adding another board-printing function. However, using the same format for the console and output file means the solution files do not match the original puzzle format.

Claude: Claude uses a separate `writeBoardToFile()` function, so the console display and the solution-file format can be different. It also has a separate function for writing the no-solution message and keeps most of the original program's structure. Claude is longer and repeats some file-handling code, but the organization makes the output easier to follow and change. Overall, I think Claude is a little easier to maintain.

F). Claude

I would choose Claude as the better starting point because it meets more of the assignment requirements while keeping the original program's overall structure. Both programs use the same backtracking method and have similar time and memory use. Claude handles the solution-file format better and also creates an output file when an input puzzle is missing or invalid. Gemini is a bit simpler, but its output formatting would need more changes. Claude could still be improved by checking file-writing errors more carefully and updating its comments. Overall, I think Claude is the stronger starting point for this assignment.

New Improved Code

```
/*
 * Program: EECS 348 Assignment 5 (revision of Assignment 4)
 * Description: Loads five Sudoku puzzles, finds every solution using recursive
 *              backtracking, displays results, and saves each puzzle's solutions.
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 * Outputs: Console results and solution1.txt through solution5.txt
 * Original author: Prabesh Sapkota
 * Original creation date: October 15, 2026
 * Revised by: Conor Rothe
 * Revision date: October 29, 2026
 * Collaborators: Original Assignment 4 author (credited above)
```

```
 * Other sources: Original code cites ChatGPT (OpenAI). This revision builds
 *                on AI-generated Code 2 (provider identified in the GenAI
 *                analysis) with additional ChatGPT (OpenAI) assistance.
 * Revision notes: Reused candidate checks, strengthened file error handling,
 *                 and preserved the original console behavior.
 */

#include <iostream> // Provides console input and output.
#include <fstream> // Provides file-reading tools.
#include <string> // Provides the string type.
#include <vector> // Stores and validates the puzzle's 81 input cells.
#include <cctype> // Provides the isspace function.
using namespace std; // Allows standard-library names without the std:: prefix.

class SudokuSolver { // Defines a class that stores and solves one Sudoku puzzle.
private: // Begins the private section of the class.
    int board[9][9]; // Stores the Sudoku grid; zero represents an empty cell.
    int solutionCount; // Counts how many complete solutions have been found.
    ofstream outputFile; // Writes each solution to the current puzzle's solutionN.txt fi
le.

    bool isSafe(int row, int col, int digit) { // Checks whether digit can go in the spec
ified cell.
        for (int j = 0; j < 9; j++) { // Checks all cells in the selected row.
            if (board[row][j] == digit) { // Detects the same digit in that row.
                return false; // Rejects a digit that already appears in the row.
            } // Ends the row-conflict condition.
        } // Ends the row scan.

        for (int i = 0; i < 9; i++) { // Checks all cells in the selected column.
            if (board[i][col] == digit) { // Detects the same digit in that column.
                return false; // Rejects a digit that already appears in the column.
            } // Ends the column-conflict condition.
        } // Ends the column scan.

        int startRow = (row / 3) * 3; // Finds the first row of the cell's 3-by-3 block.
        int startCol = (col / 3) * 3; // Finds the first column of the cell's 3-by-3 bloc
k.

        for (int i = startRow; i < startRow + 3; i++) { // Visits the three rows of the b
lock.
            for (int j = startCol; j < startCol + 3; j++) { // Visits the three columns o
f the block.
                if (board[i][j] == digit) { // Detects the same digit inside the block.
                    return false; // Rejects a digit that already appears in the block.
                } // Ends the block-conflict condition.
            } // Ends the inner block scan.
        } // Ends the outer block scan.

        return true; // Confirms that the digit follows all Sudoku placement rules.
    } // Ends isSafe.
```

```
    void printBoard() const { // Prints the current Sudoku grid without modifying it.
        for (int i = 0; i < 9; i++) { // Visits each row of the board.
            if (i % 3 == 0) { // Checks whether a new group of three rows begins.
                cout << "+-------+-------+-------+" << endl; // Prints the horizontal gri
d border.
            } // Ends the row-group condition.

            for (int j = 0; j < 9; j++) { // Visits each cell in the current row.
                if (j % 3 == 0) { // Checks whether a new group of three columns begins.
                    cout << "| "; // Prints the left boundary of a 3-column group.
                } // Ends the column-group condition.

                if (board[i][j] == 0) { // Checks whether the current cell is empty.
                    cout << "_ "; // Displays an underscore for an empty cell.
                } else { // Handles a cell containing a digit.
                    cout << board[i][j] << " "; // Displays the digit stored in the cell.
                } // Ends the empty/non-empty condition.
            } // Ends the scan of the current row.

            cout << "|" << endl; // Prints the right boundary and starts a new output lin
e.
        } // Ends the scan of all rows.

        cout << "+-------+-------+-------+" << endl; // Prints the bottom border of the g
rid.
    } // Ends printBoard.

    void writeBoardToFile() { // Writes the current board to the output file in the input
-file format.
        for (int i = 0; i < 9; i++) { // Visits each row of the board.
            for (int j = 0; j < 9; j++) { // Visits each cell in the current row.
                if (j > 0) { // Checks whether a separator is needed before this cell.
                    outputFile << " "; // Separates cells with a single space, as in the
input files.
                } // Ends the separator condition.

                if (board[i][j] == 0) { // Checks whether the current cell is empty.
                    outputFile << "_"; // Writes an underscore for an empty cell, matchin
g the input format.
                } else { // Handles a cell containing a digit.
                    outputFile << board[i][j]; // Writes the digit stored in the cell.
                } // Ends the empty/non-empty condition.
            } // Ends the scan of the current row.

            outputFile << "\n"; // Ends the current row of the output file.
        } // Ends the scan of all rows.
    } // Ends writeBoardToFile.

    // Revised with ChatGPT: keep the legal digits found during the MRV scan.
    // This avoids checking those digits a second time in solveRecursive.
```

```
bool findBestEmptyCell(int& bestRow, int& bestCol, unsigned int& bestMask) {
    int fewestCandidates = 10; // No cell can have more than nine options.
    bestRow = -1; // -1 means we have not found an empty cell yet.
    bestCol = -1;
    bestMask = 0;

    for (int row = 0; row < 9; row++) { // Search all rows.
        for (int col = 0; col < 9; col++) { // Search each cell in the row.
            if (board[row][col] != 0) { // Ignore digits that are already fixed.
                continue;
            }

            int candidateCount = 0;
            unsigned int candidateMask = 0; // Each bit represents a legal digit.
            for (int digit = 1; digit <= 9; digit++) {
                if (isSafe(row, col, digit)) { // Use the original rule checker.
                    candidateCount++;
                    candidateMask |= (1U << digit); // Remember this legal option.
                }
            }

            if (candidateCount == 0) { // This branch cannot lead to a solution.
                bestRow = row;
                bestCol = col;
                bestMask = 0;
                return true;
            }

            if (candidateCount < fewestCandidates) { // Use the most restricted cell.
                fewestCandidates = candidateCount;
                bestRow = row;
                bestCol = col;
                bestMask = candidateMask; // Save its legal digits for recursion.
                if (fewestCandidates == 1) {
                    return true; // We cannot improve on one legal choice.
                }
            }
        }
    }
    return bestRow != -1; // False only when all cells have been filled.
} // Ends findBestEmptyCell.

void solveRecursive() { // Original backtracking search with saved legal digits.
    int emptyRow = -1;
    int emptyCol = -1;
    unsigned int candidates = 0;

    if (!findBestEmptyCell(emptyRow, emptyCol, candidates)) { // Puzzle is complete.
        solutionCount++;
        cout << "\nSolution " << solutionCount << ":" << endl;
        printBoard(); // Preserve Assignment 4's console display.
```

```
            if (solutionCount > 1) {
                outputFile << "\n"; // Separate solutions in the text file.
            }
            writeBoardToFile(); // Save this solution without keeping all boards.
            return; // Keep backtracking to find any other solutions.
        }

        if (candidates == 0) { // No digit fits the chosen cell.
            return; // Stop this unsuccessful branch immediately.
        }

        for (int digit = 1; digit <= 9; digit++) {
            if ((candidates & (1U << digit)) == 0) { // Skip forbidden digits.
                continue;
            }
            board[emptyRow][emptyCol] = digit; // Try a saved legal digit.
            solveRecursive(); // Search from the updated board.
            board[emptyRow][emptyCol] = 0; // Undo the move before trying another.
        }
    } // Ends solveRecursive.

public: // Begins the public section of the class.
    SudokuSolver() : solutionCount(0) { // Initializes the solution counter when an objec
t is created.
        for (int row = 0; row < 9; row++) { // Visits each row during initialization.
            for (int col = 0; col < 9; col++) { // Visits each column during initializati
on.
                board[row][col] = 0; // Marks every cell as empty.
            } // Ends the initialization of one row.
        } // Ends board initialization.
    } // Ends the constructor.

    bool loadPuzzle(const string& filename) { // Loads and validates the basic format of
a puzzle file.
        ifstream inputFile(filename); // Opens the specified file for reading.

        if (!inputFile.is_open()) { // Checks whether the file opened successfully.
            cerr << "Error: Could not open " << filename << endl; // Reports a missing or
inaccessible file.
            return false; // Stops loading when the file cannot be opened.
        } // Ends the file-open check.

        vector<char> cells; // Temporarily stores all 81 cell symbols before changing the
board.
        char ch; // Stores each character read from the file.

        while (inputFile.get(ch)) { // Reads the file one character at a time.
            if ((ch >= '1' && ch <= '9') || ch == '_') { // Accepts digits one through ni
ne and underscores.
                cells.push_back(ch); // Saves a valid Sudoku cell symbol.
```

```
            } else if (ch == '0') { // Allows zero as another way to mark an empty cell.
                cells.push_back('_'); // Normalizes zero to the underscore blank marker.
            } else if (!isspace(static_cast<unsigned char>(ch))) { // Rejects unexpected
non-whitespace characters.
                cerr << "Error: Invalid character in " << filename << endl; // Reports an
unexpected character.
                return false; // Stops loading a malformed puzzle file.
            } // Ends the character-validation branches.
        } // Ends the file-reading loop.

        if (cells.size() != 81) { // Requires exactly 81 cell symbols for a 9-by-9 puzzl
e.
            cerr << "Error: " << filename << " must contain exactly 81 Sudoku cells." <<
endl; // Reports the incorrect cell count.
            return false; // Rejects a puzzle with too few or too many cells.
        } // Ends the cell-count check.

        for (int row = 0; row < 9; row++) { // Copies the validated symbols into each boa
rd row.
            for (int col = 0; col < 9; col++) { // Copies the symbols into each board col
umn.
                char cell = cells[row * 9 + col]; // Retrieves the symbol at the current
linear position.

                if (cell == '_') { // Checks whether the symbol represents a blank cell.
                    board[row][col] = 0; // Stores blank cells internally as zero.
                } else { // Handles a symbol containing a digit.
                    board[row][col] = cell - '0'; // Converts the digit character into it
s integer value.
                } // Ends the cell-conversion condition.
            } // Ends copying one row.
        } // Ends copying all 81 cells.

        return true; // Confirms that the file was read and formatted correctly.
    } // Ends loadPuzzle.

    bool isInitialBoardValid() { // Checks that the puzzle's original digits do not confl
ict.
        for (int row = 0; row < 9; row++) { // Examines each row of the starting puzzle.
            for (int col = 0; col < 9; col++) { // Examines each cell in the current row.
                if (board[row][col] == 0) { // Checks whether the current cell is blank.
                    continue; // Skips blank cells because they cannot conflict with a gi
ven digit.
                } // Ends the blank-cell condition.

                int digit = board[row][col]; // Saves the digit currently being checked.
                board[row][col] = 0; // Temporarily clears the cell so it does not confli
ct with itself.
                bool valid = isSafe(row, col, digit); // Checks the saved digit against i
ts row, column, and block.
                board[row][col] = digit; // Restores the original digit after checking i
```

```
t.

                if (!valid) { // Checks whether another given digit conflicts with this o
ne.
                    return false; // Rejects the starting puzzle if a conflict is found.
                } // Ends the conflict condition.
            } // Ends the scan of the current row.
        } // Ends the scan of the starting board.

        return true; // Confirms that all given digits are mutually consistent.
    } // Ends isInitialBoardValid.

    // Revised with ChatGPT: report whether the failure message was saved.
    bool writeNoSolution(const string& outputFilename) {
        ofstream failFile(outputFilename.c_str()); // Replace any stale output.
        if (!failFile.is_open()) {
            cerr << "Error: Could not write " << outputFilename << endl;
            return false;
        }
        failFile << "No solution found.\n"; // Exact required message.
        failFile.close(); // Flush the message and check for a disk error.
        if (!failFile) {
            cerr << "Error: Failed to finish writing " << outputFilename << endl;
            return false;
        }
        return true;
    } // Ends writeNoSolution.

    // Revised with ChatGPT: stop early if the solution file cannot be written.
    bool solve(const string& filename, const string& outputFilename) {
        cout << "\n========================================" << endl;
        cout << "Puzzle file: " << filename << endl;
        cout << "========================================" << endl;
        cout << "\nOriginal puzzle:" << endl;
        printBoard(); // Keep the original console presentation.

        if (!isInitialBoardValid()) { // Don't search puzzles with conflicting clues.
            cout << "\nNo solution found: the original puzzle has conflicting digits." <<
endl;
            return writeNoSolution(outputFilename);
        }

        solutionCount = 0; // Start each puzzle with a new solution count.
        outputFile.open(outputFilename.c_str());
        if (!outputFile.is_open()) { // Don't continue if required output is unavailable.
            cerr << "Error: Could not write " << outputFilename << endl;
            return false;
        }

        solveRecursive(); // Find and write every solution using backtracking.
        if (solutionCount == 0) {
```

```
            cout << "\nNo solution found." << endl;
            outputFile << "No solution found.\n";
        } else {
            cout << "\nTotal solutions found: " << solutionCount << endl;
            outputFile << "\nSolutions found: " << solutionCount << "\n";
        }

        outputFile.close(); // Flush output before reporting success.
        if (!outputFile) { // Catch write or close failures.
            cerr << "Error: Failed to finish writing " << outputFilename << endl;
            return false;
        }
        return true;
    } // Ends solve.
}; // Ends the SudokuSolver class definition.

int main() { // Processes all five required puzzle files.
    const string filenames[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.tx
t", "puzzle5.txt"};
    bool allFilesProcessed = true; // Track errors while still checking all puzzles.

    for (int i = 0; i < 5; i++) {
        const string& filename = filenames[i];
        string outputFilename = "solution" + to_string(i + 1) + ".txt";
        SudokuSolver solver; // Keep each puzzle's board and count separate.

        if (solver.loadPuzzle(filename)) {
            if (!solver.solve(filename, outputFilename)) {
                allFilesProcessed = false; // The output file could not be saved.
            }
        } else {
            // Preserve Code 2's output-file behavior for missing or bad inputs.
            // Also report a process error because the input itself was invalid.
            if (!solver.writeNoSolution(outputFilename)) {
                cerr << "Error: Unable to create " << outputFilename << endl;
            }
            allFilesProcessed = false;
        }
    }

    return allFilesProcessed ? 0 : 1; // Nonzero means at least one file failed.
} // Ends main.
```

G). Improvements Made

Correctness: After choosing Claude's version, the original code could keep running even if it could not open th e output file. I changed it to report the problem and stop solving that puzzle. I also added checks to make sure the files were written successfully and made the program return an error if a file could not be processed.

Execution Time: I updated the functions that choose the next empty cell and solve the puzzle to save possible digits in a bitmask. Before this change, the program checked which digits were valid when picking a cell and then checked

them again during recursion. Now it reuses the options it already found, which cuts down on repeated checks. The worst-case time complexity is still O(9^E).

Space Complexity: The program still writes solutions straight to the output files instead of storing them all in memory. The bitmask uses only a small amount of extra space during recursion. Since a Sudoku board is always 9×9, the program's working space stays O(1) for a standard puzzle. This change keeps the program memory-efficient rather than reducing its original space complexity.

Maintainability: I updated the program's opening comments to give credit to the original author and explain the revisions and AI assistance. I also added comments for the new logic and organized the file checks so errors are easier to understand. I kept the SudokuSolver class and most of the original functions, so the revised program still follows the structure of the original code and is easier to work with.